LAB 2- github link

# Shiv Nadar University Chennai

**CS3807 – Deep Learning Laboratory**

**Experiment 2**

**Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification**

| | | |
|---|---|---|
| Degree & Branch | B.Tech Artificial Intelligence & Data Science | Semester V |
| Subject Code & Name | CS3807 – Deep Learning Laboratory | AY: 2026–27 |

## 1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

## 2. Learning Outcomes

Students will be able to:

1. Explain the architecture of an MLP.
2. Preprocess image datasets.
3. Build and train an MLP.
4. Evaluate classification performance.
5. Perform automated hyperparameter optimization.
6. Recommend the best model configuration.

## 3. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \qquad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = -\sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

## 4. Dataset

Dataset: Fashion-MNIST

Training Images: 60,000

Testing Images: 10,000

Classes: 10

Image Size: $28 \times 28$

## 5. Image Preprocessing

Fashion-MNIST images are stored as $28 \times 28$ matrices. Dense layers require a one-dimensional feature vector.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

Perform the following preprocessing:

1. Load Fashion-MNIST.
2. Display sample images.
3. Flatten images.
4. Normalize pixels to [0,1].
5. Convert labels into one-hot vectors.

## 6. Experimental Procedure

**Task 1: Dataset Exploration**

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

**Expected Output:** Dataset summary and sample images.

**Task 2: Data Preprocessing**

- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

**Task 3: Model Construction**

Construct an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display `model.summary()`.

**Task 4: Model Training**

Compile using

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 32.

**Task 5: Model Evaluation**

Compute

- Accuracy

- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

## 7. Hyperparameter Optimization

Instead of manually selecting hyperparameters, students shall perform automated hyperparameter optimization using either **GridSearchCV** or **RandomizedSearchCV** (recommended) together with the SciKeras wrapper.

Optimize the following search space.

| Hyperparameter | Candidate Values |
|---|---|
| Hidden Layers | 1, 2, 3 |
| Hidden Neurons | 32, 64, 128, 256 |
| Learning Rate | 0.1, 0.01, 0.001 |
| Batch Size | 16, 32, 64, 128 |
| Epochs | 10, 20, 30 |
| Optimizer | SGD, Adam, RMSProp |
| Activation Function | ReLU, Tanh, Sigmoid |
| Dropout Rate (Optional) | 0.0, 0.2, 0.5 |

### Tasks

1. Build a baseline MLP model.
2. Define the hyperparameter search space.
3. Perform Grid Search or Randomized Search using 5-fold cross-validation.
4. Record the best hyperparameter combination.
5. Retrain the model using the optimized hyperparameters.
6. Evaluate the optimized model on the testing dataset.
7. Compare the optimized model with the baseline model.

## 8. Mandatory Plots

Generate the following plots.

1. Sample Images
2. Class Distribution
3. Training Accuracy vs Epoch
4. Validation Accuracy vs Epoch
5. Training Loss vs Epoch
6. Validation Loss vs Epoch
7. Confusion Matrix
8. Hyperparameter Search Results
9. Best Model Accuracy Comparison

Write a 2–3 line inference for each plot.

## 9. Results

**Best Hyperparameters**

| Hyperparameter | Value |
|---|---|
| Hidden Layers | 2 |
| Hidden Neurons | 256 |
| Learning Rate | 0.001 |
| Batch Size | 32 |
| Optimizer | adam |
| Activation Function | sigmoid |
| Epochs | 30 |
| Dropout | 0.0 |
| Cross-validation Accuracy | 0.8477 |
| Testing Accuracy | 0.8885 |

**Performance Comparison**

| Metric | Baseline | Optimized |
|---|---|---|
| Accuracy | 0.8825 | 0.8886 |
| Precision | 0.8848 | 0.8900 |
| Recall | 0.8825 | 0.8886 |
| F1-score | 0.8830 | 0.8882 |
| Training Time (s) | 167.59 | 342.68 |

## 10. Plots with inference


Figure 1: sample images

the figure shows the grayscale visualization of the first 10 images from the dataset. all the images has a 28x28 pixel resolution. while the image is relatively blurred, it contains enough information to preserve the structural image of the image and the object

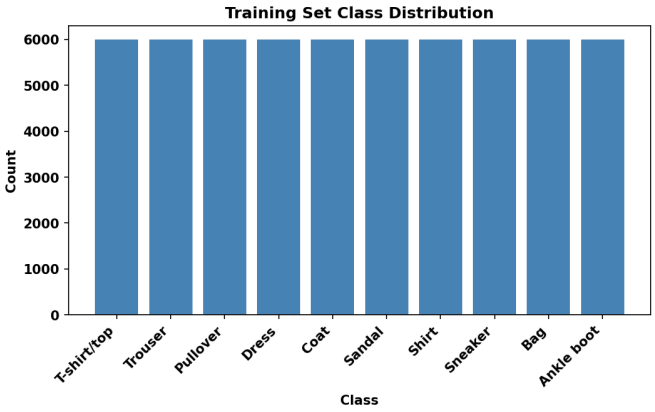


Figure 2: class distribution

from the above class distribution, we see that all the 10 classes have equal number of training images which is 6000. this makes the dataset balanced with a no class being over-represented and the model can learn each class equally.

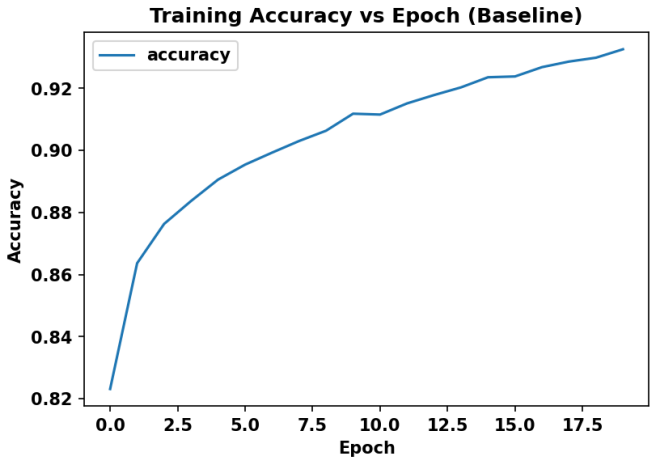


Figure 3: training accuracy

the graph shown above depicts the improvement of the training accuracy through out the training process. the model initially has a .82 accuracy which has a steep rise to around .87 and a steady increase until it reached .93 the absence of drops and oscillation insinuates that the model converged smoothly.

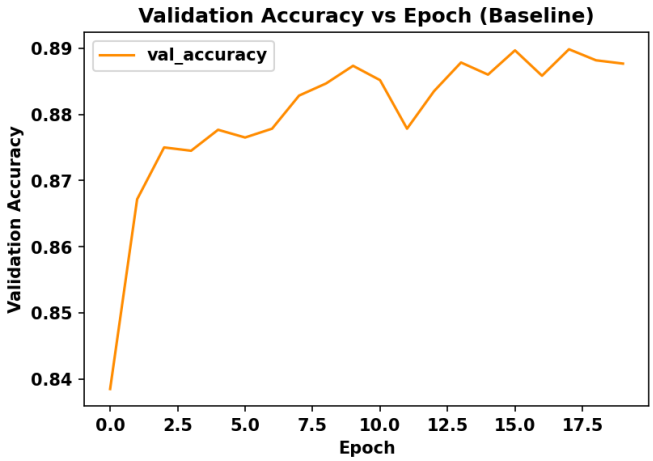


Figure 4: val accuracy

the validation accuracy graph is slightly different from the training accuracy graph, which is mainly due to the unseen data during the training. Hence, the validation reached to only .89 in accuracy. since there is only a slight difference in the accuracy from training, the model doesn't suffer from severe overfitting and has a good generalization

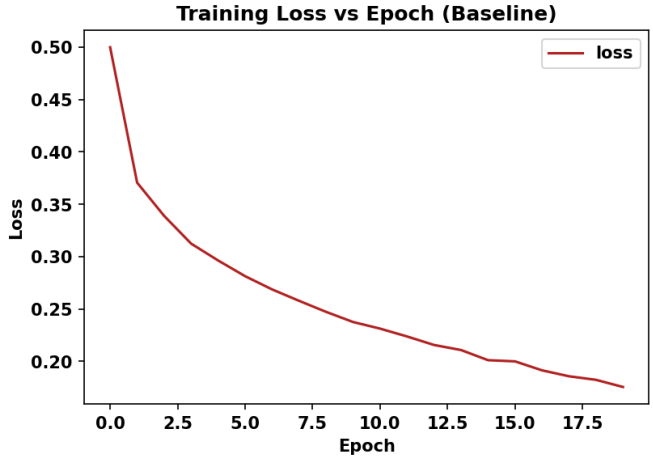


Figure 5: training loss

The graph shows a rapid decrease from 0.50 to .35 and then slowly decrease to .18 by the final epoch. The slower decline shows that the model is converging towards an optimal solution

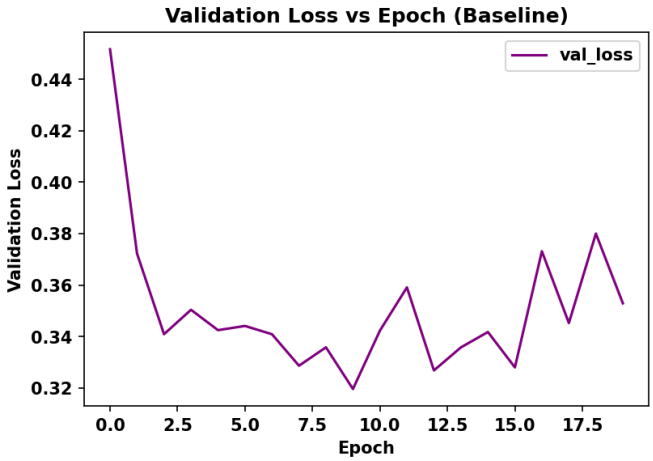


Figure 6: validation loss

The validation loss initially decrease from 0.41 and to 0.32, however, in the later epoch, we see a rise in the validation loss and reach to 0.41 which shows that the model is beginning to overfit. hence, the early stopping around epoch 10-13 will give better performance

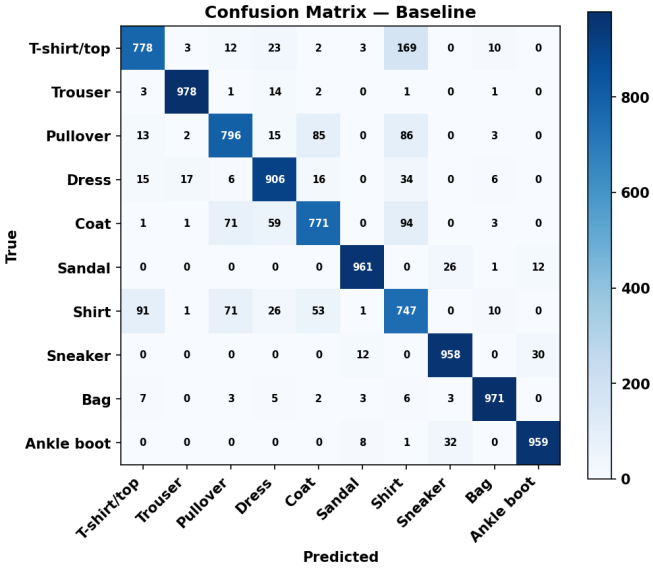


Figure 7: confusion matrix

the confusion matrix shows a summary of the model classification performance. This demonstrates that the baseline CNN model performs well on classes like footwear and accessories while it struggles with top wear. this shows that we may need deeper layers to improve the classification in the accuracy in classification of these categories

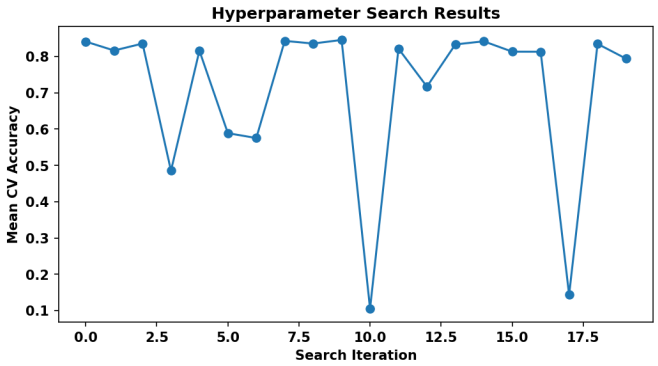


Figure 8: hyperparameter search results

The graph shows the mean cross validation accuracy obtained from the multiple hyperparameter configuration that was run to find the best hyperparameter results, which shows that a mix of good and poor performances.

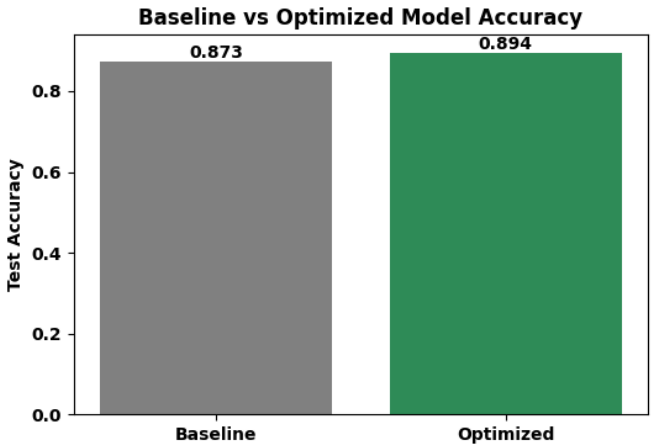


Figure 9: accuracy comparison

the figure shows the test accuracy of the test data with respect to the baseline CNN model and optimized hyperparameter CNN model. the reduction in accuracy depicts overfitting of the variables Answer the following:

## 10. Dicsussion

Answer the following:

1. Which optimization method was used? The model was optimized using Randomized Search Cross-Validation (RandomizedSearchCV)

2. Which hyperparameters were selected?
   Optimizer: Adam Learning Rate: 0.001 Hidden Layers: 2 Neurons per Hidden Layer: 256 Activation Function: Sigmoid Dropout Rate: 0.0 (No Dropout) Epochs: 30 Batch Size: 32

3. Did optimization improve performance?
   yes, the optimization has improved the performance but it only by 2 percent

4. Which hyperparameter had the greatest impact?
   the hidden layer architecture had the biggest impact in the improvement of accuracy

5. Compare Grid Search and Randomized Search.
   Compare grid search takes longer execution time but looks through all possible hyperparameter combination, which is suitable whe the number of hyper parameters are less. The randomized search has less computationally expensive and useful when we need to go through multiple hyperparameter combination while still giving a comparable performance

6. Recommend the best MLP configuration.
   Best Hyperparameter Configuration: Hidden Layers – 2; Neurons per Hidden Layer – 256; Activation Function – Sigmoid; Optimizer – Adam; Learning Rate – 0.001; Dropout Rate – 0.0 (No Dropout); Batch Size – 32; Epochs – 30; Best 5-Fold Cross-Validation Accuracy – 84.80

# 10. Additional task - 23.07.2026

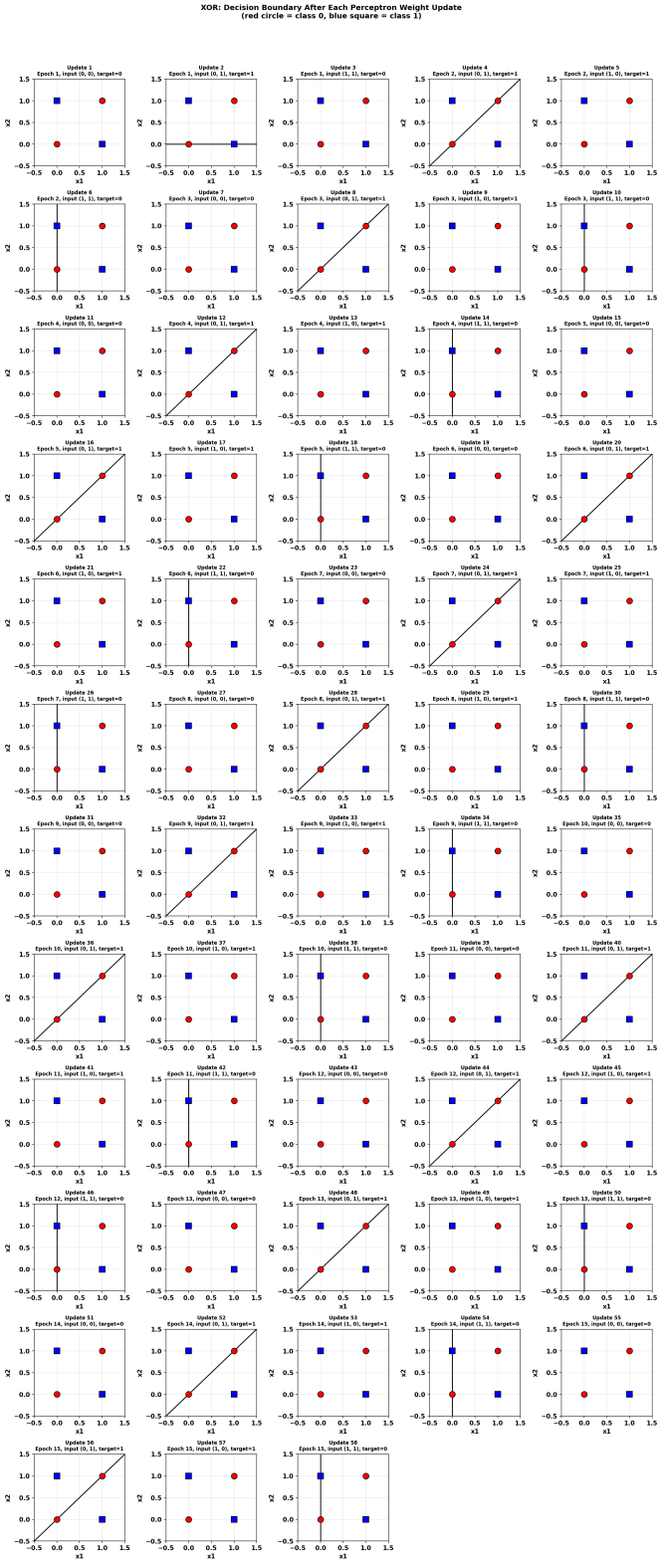


Figure 10: xor decision boundary

2. Code the perceptron learning algorithm for the XOR gate using Python. Show the weights after each update. Plot the decision boundary after each weight update using Python's built-in packages. Analyze and identify the pattern that prevents the algorithm from converging. (K4)

Initially, the weights are initialized to 0 and then it gets updated as it learns using the weight update rule. After 3-4 epoch, the values of the weights didnt changed to anything new and cycled through the 4 sets of weights that were previously learned where each one gave a different decision boundary: horizontal, vertical, diagonal.

we can see that the decision boundary is cycling ever 8 images (8 epochs). This happened because the 2 points of each class are diagonally opposite to each other, hence there is no straight line that was separate these 2 classes. We can see that, while it was a able to classify one point correctly, it simultaneously misclassified another point, hence the convergence never occurs.

this exactly proves the perceptron convergence theorem, where it states that the perceptron learning will learn by adjusting the weights and bias (the parameters of the algorithm) and converge if and only if the data is linearly separable. Hence why the perceptron learning algorithm cannot solve the XOR problem and it requires Multi layer perceptron with atleast one hidden layer to be able to solve the non linear decision boundary.

## 11. Conclusion

The experiment shows the multi-layer perceptron performs well for the fashion-MNIST dataset consisting of image data which had a balanced distribution of all classes. While the baseline model works well, the hyperparameter optimization increased its accuracy. Overall, the model was reliable and an efficient solution for the image dataset.

## 12. Report Contents

Include Objective, Theory, Dataset, Experimental Procedure, Source Code, Hyperparameter Optimization, Results, Plots with Inference, Discussion, Conclusion and References.

## 12. References


1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.


LAB 2- github link